01 - From a problem to a requirement

Teacher reasoning and follow-up

Separate inputs, processing and outputs, then write a requirement you can test.

8 min reading, 12 min modelling, 20 min application, 10 min return, 5 min exit.

1A-1B

5 is input; multiply by 3 is processing; Total: $15 is output; allowed quantities are an assumption/domain rule. Accept equivalent requirements with a measurable output and declared valid input. 'User-friendly' alone is not testable.

1C

A: input quantity, multiply by 2 then add 4, display cost; assumption every order has the charge. B: input distance, calculate 10 + distance, display position; assume valid non-negative distance. C: input label and supplied Boolean, choose the corresponding message, display it; supplied status is not a live network observation.

1D / exit

Card A gives $4 for zero and $14 for five. Badge exit gives input 5, processing 5 * 3, output $15. Accept equivalent explicit assumptions consistent with the card; do not silently waive the charge.

Reteach / extension

If input and output are reversed, use three labelled boxes and trace 2 badges. Ask for a changed test quantity. A correct number without explaining the transformation is incomplete reasoning.

02 - Values, types and a saved program

Teacher reasoning and follow-up

Run a saved sequence, distinguish text from numbers and predict simple calculations.

7 min reading, 13 min modelling, 20 min application, 10 min return, 5 min exit.

2A

Identifier str preserves zeros; count int; measured length float; paid bool. 'It looks numeric' is not enough to justify storing an identifier as a number.

2B

12; '444'; 4; 1; 14; 20. The string result is text, not numeric 444. // is whole-number floor division; % is remainder for these non-negative examples. Parentheses change the evaluation order.

2C-2D

Expected totals are $6 then $8; leave actual outcomes blank until observed. Ticket rule: total = quantity * 4 + 1. Quantities 0,1,3 give 1,5,13. A different valid variable naming style is acceptable.

Exit / reteach

input returns text; converting valid numeric text enables arithmetic rather than repetition/concatenation. Identifier formatting matters and arithmetic is not meaningful for it. Optional 2 ** 3 is 8. If a new price appears not to work, inspect the actual saved/running file rather than changing the formula at random.

03 - Requirements and a realistic schedule

Teacher reasoning and follow-up

Translate a brief into measurable requirements and draw task durations with dependencies.

8 min reading, 10 min modelling, 22 min application, 10 min return, 5 min exit.

3A

Examples: calculate quantity * 2 + 4 for every allowed quantity; display the calculated total. Assumption: input is a non-negative whole number. Payment processing is excluded. For quantity 5, expected total is $14. Accept equivalent measurable requirements; appearance preferences alone are insufficient.

3B-3C

Original finishes slot 8. Revised: A1, B2, C3-6, D7-8, E9. Finish is 9. Negotiate scope/deadline/resources with a justified revised plan. Do not merely overlap tasks whose predecessor must finish first or silently remove testing.

3D

Investigate1; design2-3; build4-5; test6; evaluate7. Example requirement: for a supplied label/status, display that label and the correct online/offline message. Testing checks the output against the requirement; deleting it supplies no evidence of correctness.

Exit / reteach

Duration is time needed; deadline is required finish; dependency constrains start relative to other work. Model the chart with cards before redrawing if pupils treat bars as decoration rather than constraints. No student is assumed to have built the program merely by drawing the plan.

04 - Trace a sequence and represent a character

Teacher reasoning and follow-up

Track changing values, distinguish an expected result from an observation, and convert one character code between bases.

10 min reading, 10 min modelling, 20 min application, 10 min return, 5 min exit.

4A

Line1 quantity3; line2 price2; line3 handling4; line4 subtotal6; line5 total10; line6 output10. Previously assigned values remain unchanged. Before its assignment a name is not set, not automatically zero.

4B

Syntax fault: print("Total"). A logic fault can still run but implement the wrong formula. Do not tell students to execute a syntax-invalid file as if it were a working starter.

4C-4D

Quantity0: subtotal0, total/output4. Quantity5: subtotal10, total/output14. Actual columns are only completed after observation. B is 01000010 and 42 hexadecimal; 64+2=66 and 4*16+2=66. A code point identifies a character; UTF-8 byte count depends on its encoding, and a visible character may involve more than one code point.

Exit / reteach

Original output10. Prediction supplies an independent expectation instead of accepting whatever the code displays. If place values are weak, write the 128/64/32/16/8/4/2/1 row and add the selected positions; retain the extra zeros for the requested eight-bit form.

05 - Conditions and Boolean reasoning

Teacher reasoning and follow-up

Evaluate comparisons and combine Boolean conditions with and, or and not.

8 min reading, 12 min modelling, 20 min application, 10 min return, 5 min exit.

5A-5B

Truth rows (and,or,not A): (False,False,True), (False,True,True), (False,True,False), (True,True,False). With score7: >=5 True, ==5 False, !=5 True. Assignment changes/binds a value; it is not an equality comparison.

5C

Rows ready=False,False,True,True. The comparison battery>=30 is False,True,True,True. Equality at 30 is included.

5D-5E

First pair False and False; second pair True and False. Parentheses can change the expression's meaning. Entry expression age >= 12 and member gives False,False,True for the listed cases. Accept an equivalent explicit Boolean comparison.

Exit / reteach

For True/False (or False/True), or is True and and is False. If pupils reason from one operand only, evaluate both components before combining them. These examples use Boolean operands; do not generalise every Python and/or result to a bool for arbitrary objects.

06 - Choose exactly the intended branch

Teacher reasoning and follow-up

Use if/elif/else and trace a nested decision without executing both alternatives.

7 min reading, 13 min modelling, 20 min application, 10 min return, 5 min exit.

6A-6B

-1 selects negative/invalid; 0 selects equality/empty; 2 selects else/process. Delivery: if order_total >= 50: charge=0; else: charge=5. Expected charges 5,0,0. Assume stated valid non-negative numeric totals; do not silently add unrequested input handling.

6C

START/False prints not authorised; START/True prints ready; STOP/True prints not a start request. Inner else belongs to if authorised; outer else belongs to if command == START. Indentation is essential.

6D

if command == 'SHOW': then an inner if staff prints full report, inner else summary; outer else unknown. Outputs summary, full report, unknown for the three cases. Accept equivalent logic with the same observable behaviour; compare its structure explicitly if it is not nested.

Exit / reteach

Separate if statements each test their own condition; more than one may run. An if/elif/else chain selects at most one alternative. If pupils execute both branches, draw a path arrow and stop evaluating alternatives after a match.

07 - A reliable text command menu

Teacher reasoning and follow-up

Normalise text deliberately and separate conversion, domain validation and permission.

8 min reading, 12 min modelling, 20 min application, 10 min return, 5 min exit.

7A

' start '->START->Start requested; Stop->STOP->Stop requested; STATUS->STATUS->Status requested; whitespace-only->empty string->Unknown command; ST ART remains ST ART->Unknown command. The internal space is not removed by strip().

7B

Bind raw input, normalise to command, then compare exactly with START, STOP and STATUS in one if/elif chain, otherwise Unknown command. Do not change messages silently. No actual results are claimed before a learner runs their version.

7C

rating >= 1 and rating <= 5. int('-4') produces an integer but violates the domain. int('three') raises ValueError; knowing that is distinct from being required to write an exception handler now.

7D / exit / reteach

Normalisation adjusts representation under the stated contract; validation checks allowed values; authorisation concerns permitted actions. START can be valid text while a caller lacks permission. If unknown strings reach a valid branch, inspect exact comparisons and the final else; retest blank and internal-space cases.

08 - Test both sides of the boundary

Teacher reasoning and follow-up

Predict discriminating tests, find a boundary defect and retain failure/repair/retest evidence.

5 min reading, 10 min modelling, 25 min application, 10 min return, 5 min exit.

8A-8B

Required results for 0,1,3,5,6: False,True,True,True,False. Faulty results: False,False,True,False,False. Cases 1 and 5 expose exclusive versus inclusive comparisons; 3 passes both rules and cannot distinguish them.

8C

Repair to rating >= 1 and rating <= 5. Retest 0,1,3,5,6. Expected values can be written now; actual-before/after fields are only completed from observed runs. Retain a build/version note and the reason for repair.

8D

quantity >= 2 and quantity <= 8. Suitable tests 1 reject,2 accept,5 accept,8 accept,9 reject. Accept another valid typical value and equivalent condition with the same domain.

Exit / reteach

Original four endpoint cases: 0 reject,1 accept,5 accept,6 reject. A later threshold change needs new near-boundary values and regression checks on previous valid behaviour. If pupils copy outputs as expectations, calculate expected acceptance from the written requirement first.